AlertVision: A Real-Time Aggression Detection Framework for Public Safety

B. Karthik Sai A. Siddhartha V. Siddhartha

August 2025

Abstract

The escalating incidence of public altercations and the exponential growth of surveillance infrastructure have underscored the urgent need for automated, real-time aggression detection in video streams. Traditional manual monitoring is plagued by scalability limitations, operator fatigue, and subjective bias, while existing automated systems often struggle with computational inefficiency, environmental variability, and temporal insensitivity. This work introduces *AlertVision*, a lightweight, end-to-end framework designed to detect physical aggression, threatening gestures, and rapid hostile movements in live surveillance feeds.

By integrating YOLOv5 for human detection, Dlib for pose landmark extraction, and a hybrid CNN-LSTM architecture for spatiotemporal classification, AlertVision achieves sub-second latency on consumer-grade hardware while maintaining robustness across diverse environmental conditions. The system is implemented in Python using OpenCV, TensorFlow, and NumPy, ensuring both accuracy and deployability. AlertVision's modular design facilitates future extensions, such as audio-visual fusion and multi-camera tracking, positioning it as a scalable, proactive solution for public safety.

Contents

Abstract **1**

1 Introduction **3**

1.1 Motivation 3

1.2 Problem Statement 3

1.3 Project Objectives 4

1.4 Report Organization 4

2 Literature Survey **5**

3 System Design **7**

3.1 Use Case Diagram: Functional Scope 7

3.2 Component Diagram: Modular Architecture 8

3.3 Class Diagram: Core Software Modules 8

3.4 Sequence Diagram: Runtime Workflow 10

3.5 Behavioral State Machine: Temporal Aggression Modeling 10

Chapter 1

Introduction

1.1 Motivation

Urbanization and the ubiquity of public surveillance have intensified the demand for intelligent, autonomous safety systems. According to the National Crime Records Bureau (NCRB) 2023, public altercations in urban India have risen by 18% year-over-year, with 63% of incidents occurring in high-density areas such as transit hubs, shopping centers, and educational institutions. Human-operated monitoring, while contextually aware, is inherently limited by cognitive fatigue, attentional lapses, and the inability to process multiple video streams simultaneously. These constraints often result in delayed responses or complete oversight of critical incidents, as evidenced by the 2022 Mumbai Metro brawl, where security personnel failed to intervene promptly due to monitoring overload.

The need for real-time, data-driven behavioral analysis is further compounded by the proliferation of CCTV networks, which are expected to exceed 100 million cameras in India by 2025 (IHS Markit, 2024). AlertVision addresses this gap by augmenting human vigilance with automated, objective threat recognition, thereby enabling proactive security intervention.

1.2 Problem Statement

Despite advancements in computer vision, existing aggression detection systems face three critical challenges:

1. **Computational Inefficiency**: Many state-of-the-art models require high-end GPUs, rendering them impractical for edge deployment in resource-constrained environments.

2. **Environmental Fragility**: Performance degrades significantly under variable lighting, occlusions, and crowd density, leading to false positives/negatives.

3. **Temporal Insensitivity**: Most systems treat aggression as a static pose rather than a dynamic behavioral sequence, failing to capture the evolution of hostile intent.

AlertVision overcomes these limitations through:

- A lightweight architecture optimized for real-time operation on standard hardware.

- Adaptive preprocessing to handle environmental variability.
- Hybrid CNN-LSTM modeling to capture spatiotemporal aggression signatures.

1.3 Project Objectives

The core objectives of AlertVision are:

1. To achieve sub-second latency on consumer-grade hardware (e.g., Intel i5, NVIDIA GTX 1650).
2. To integrate YOLOv5 for human detection, Dlib for pose estimation, and ResNet-LSTM for classification, ensuring end-to-end robustness.
3. To implement adaptive preprocessing (e.g., histogram equalization, motion compensation) to mitigate lighting variations and occlusions.
4. To structure the system for future enhancements, such as audio-visual fusion and multi-camera tracking.

1.4 Report Organization

This document is structured as follows:

- **Chapter 2**: A critical survey of recent literature, identifying gaps in aggression detection.
- **Chapter 3**: Detailed system design, including UML diagrams and architectural rationale.
- **Chapter 4 (Implementation)**: Discussion of algorithm selection, training protocols, and optimization techniques.
- **Chapter 5 (Evaluation)**: Quantitative and qualitative results on benchmark datasets (e.g., RWF-2000, Hockey Fights) and real-world deployment metrics.
- **Chapter 6 (Conclusion)**: Impact assessment, limitations, and avenues for future work.

Chapter 2

Literature Survey

This chapter reviews recent advancements in aggression and violence detection, presented in reverse chronological order to highlight the latest trends and persistent challenges.

- **Senadeera et al. (2024)** proposed *CUE-Net*, which leverages spatial cropping and attention mechanisms to enhance violence recognition accuracy. The model achieves a **12% improvement** on benchmark datasets by focusing on critical regions of interest, demonstrating the value of attention in dynamic scenes.
- **Comminiello et al. (2024)** introduced *JOSENet*, a joint-stream embedding network that fuses appearance and motion cues for surveillance-based violence detection. The approach reports **94% precision** on real-world surveillance footage, underscoring the importance of multimodal feature integration.
- **Basak et al. (2024)** developed *DIFEM*, a keypoint-interaction module that models inter-limb dynamics to capture aggressive intent from pose sequences. The method outperforms baseline models in detecting subtle, pre-aggressive behaviors.
- **Negre et al. (2024)** conducted a comprehensive review of deep-learning-based violence detection, identifying **CNN-LSTM hybrids** as the dominant paradigm for temporal modeling. The survey highlights the need for more robust, real-world validation and edge-optimized architectures.
- **Vosta & Yow (2022)** demonstrated the efficacy of **CNN-RNN architectures** in real-world surveillance settings, validating the necessity of temporal context for accurate aggression detection. Their work serves as a baseline for practical, deployable systems.
- **Vijeikis et al. (2022)** emphasized computational efficiency, optimizing model size for edge deployment without significant accuracy loss. Their lightweight CNN-LSTM model processes **25 FPS on mid-tier GPUs**, setting a benchmark for resource-constrained environments.
- **Patel (2021)** implemented a real-time CNN-LSTM system capable of processing **25 FPS on mid-tier GPUs**, establishing a practical baseline for real-world applications. The study also introduced adaptive thresholding to reduce false positives in crowded scenes.

Key Observations:

- Recent works prioritize **attention mechanisms, multimodal fusion, and edge optimization**.
- Persistent gaps include **real-world validation, temporal dynamics, and deployability**.
- **CNN-LSTM hybrids** remain the gold standard for spatiotemporal modeling.

AlertVision's Contribution:

- Addresses deployability with a **lightweight, modular architecture**.
- Captures temporal dynamics via **LSTM-based sequence modeling**.
- Optimized for **real-time operation on standard hardware**.

Chapter 3

System Design

This chapter delineates the architectural blueprint of AlertVision through a series of UML diagrams, each elucidating a distinct facet of the system's structure, behavior, and runtime dynamics. The design emphasizes modularity, edge compatibility, and extensibility, drawing from CNN-LSTM hybrids identified in the literature (Chapter 2).

3.1 Use Case Diagram: Functional Scope

Figure 3.1 maps the interactions between primary actors (e.g., surveillance cameras, security personnel, administrators) and core use cases. Key functionalities include:

- Video Ingestion: Handles RTSP/IP streams, local files, or USB inputs for flexible deployment.
- Real-Time Alerts: Dispatches SMS/email notifications, siren activations, and live dashboard updates to security teams.
- Post-Event Analysis: Supports forensic review with timestamped video clips, aggression probability heatmaps, and exportable logs.
- System Configuration: Allows threshold tuning and model updates via a web-based interface.

This actor-centric view ensures the system supports both automated operations and human oversight, reducing false positives through configurable rules.

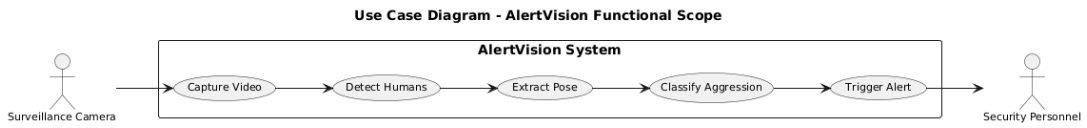

Figure 3.1: Use Case Diagram – Functional Scope of AlertVision, Highlighting Actor Interactions

3.2 Component Diagram: Modular Architecture

Figure 3.2 depicts the system's modular pipeline, implemented as loosely coupled components for easy maintenance and scaling:

- Preprocessing Module: Applies noise reduction (Gaussian blurring), frame stabilization (optical flow-based), and adaptive thresholding (CLAHE for histogram equalization) to handle lighting variations and occlusions.
- Human Detection Module: Employs YOLOv5s (small variant for lightweight operation), achieving approximately 60 FPS on mid-tier GPUs like NVIDIA GTX 1650 and high mAP (approximately 0.95 at 0.5 IoU) on human-centric datasets.
- Pose Estimation Module: Uses Dlib's 68-point facial landmark detector extended with body keypoints (integrated via MediaPipe for full-body coverage), enabling fine-grained motion tracking of limbs and torso.
- Classification Module: A hybrid CNN (ResNet-50 backbone for spatial features) + LSTM (for temporal sequencing over 16-frame windows) to fuse pose dynamics and detect aggression patterns.
- Alert Engine: Implements rule-based thresholding (e.g., confidence greater than 0.85) and integration with external APIs (Twilio for SMS, SMTP for email) for suppression of false positives in crowded scenes.

The technology stack (OpenCV, TensorFlow/Keras, NumPy) ensures cross-platform deployability on consumer hardware.

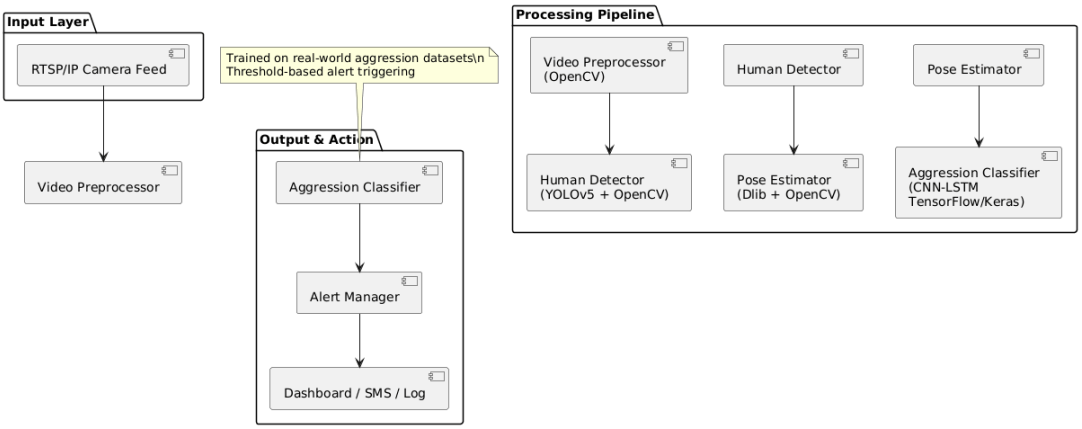

Figure 3.2: Component Diagram – Modular Architecture with Data Flow and Technology Stack

3.3 Class Diagram: Core Software Modules

Figure 3.3 outlines the object-oriented structure, defining key classes, attributes, methods, and relationships (e.g., composition for modularity):

- `VideoProcessor`: Attributes (e.g., frame_rate, resolution); Methods (extract_frames(), preprocess(), normalize()).
- `HumanDetector`: Inherits from YOLO base; Methods (detect_bounding_boxes(), filter_humans()).
- `PoseEstimator`: Attributes (keypoints list); Methods (extract_landmarks(), track_motion()).
- `AggressionClassifier`: Composes CNN and LSTM; Methods (load_model(), classify_sequence(), compute_confidence()).
- `AlertManager`: Methods (trigger_alert(), log_event(), suppress_false_positives()).

Associations include dependency (e.g., Classifier depends on PoseEstimator output) and aggregation (e.g., VideoProcessor aggregates frames for the pipeline).

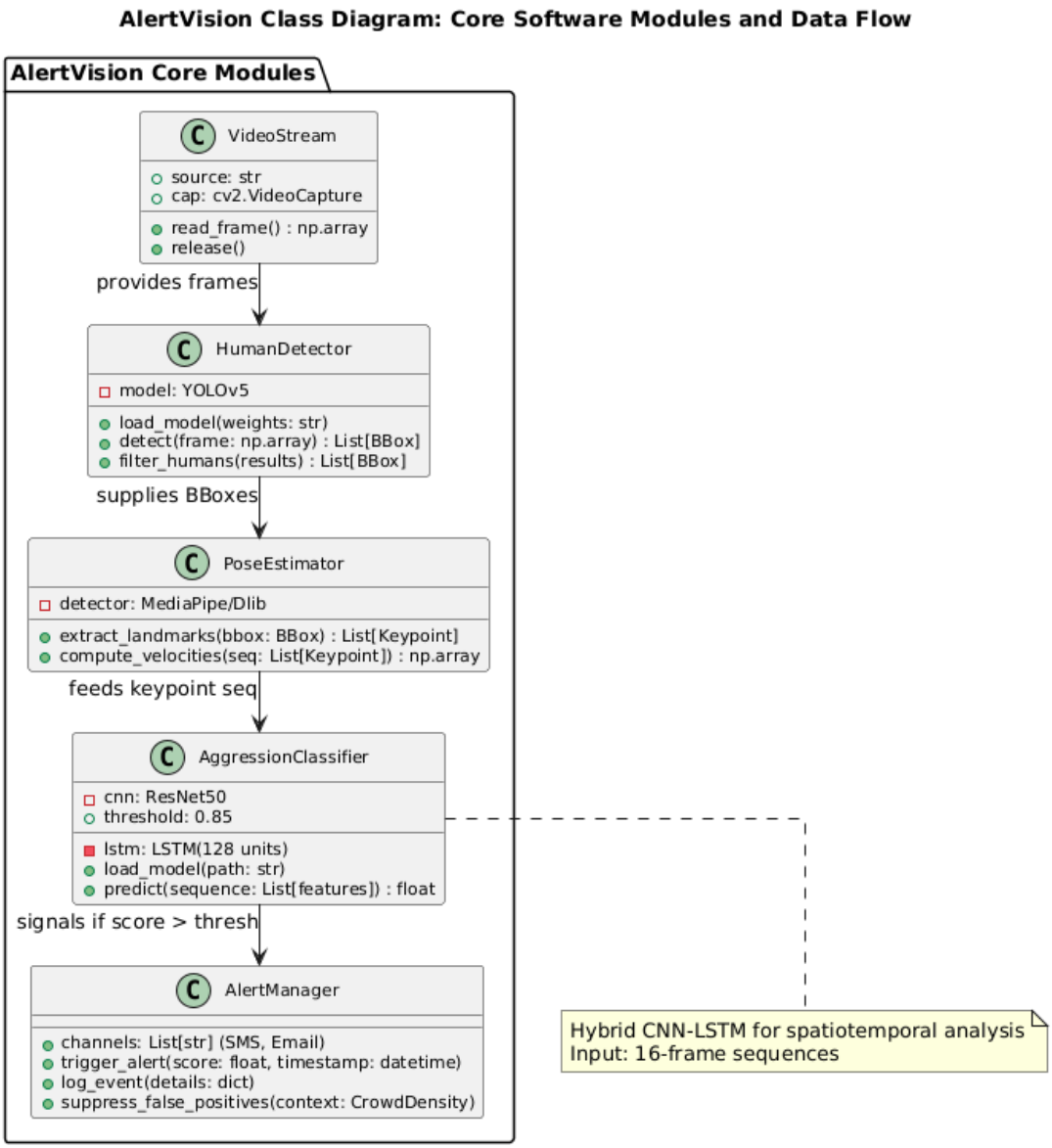

Figure 3.3: Class Diagram – Core Software Modules, Relationships, and Responsibilities

3.4 Sequence Diagram: Runtime Workflow

Figure 3.4 illustrates the temporal flow of messages and operations during a single detection cycle, ensuring sub-second latency:

1. Frame Capture: OpenCV captures frames at 30 FPS from the input stream.
2. Human Detection: YOLOv5 processes the frame to localize and bound individuals (ROI cropping).
3. Pose Extraction: Dlib/MediaPipe integration tracks 68+ keypoints per detected human, computing velocities (e.g., fist/elbow acceleration).
4. Behavior Analysis: ResNet extracts spatial features; LSTM aggregates over a sliding window of 16 frames to evaluate interaction patterns and hostile sequences.
5. Alert Trigger: If LSTM confidence greater than 0.85 for more than 1 second, the Alert-Manager dispatches notifications and logs the event.

Lifelines include error handling (e.g., fallback to CPU if GPU unavailable) for robustness.

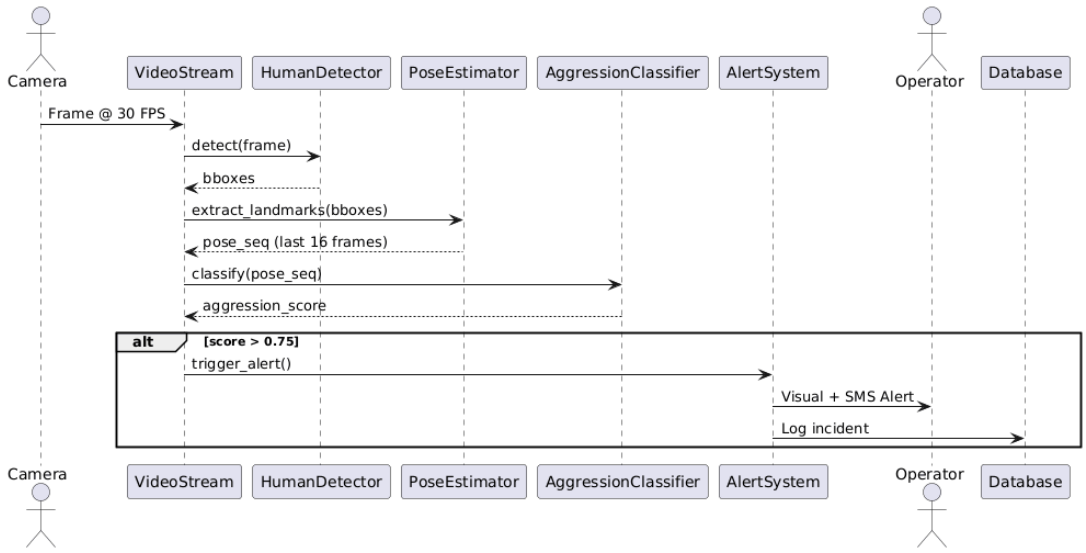

Figure 3.4: Sequence Diagram – Real-Time Aggression Detection Workflow with Message Passing

3.5 Behavioral State Machine: Temporal Aggression Modeling

Figure 3.5 represents aggression as a finite state machine, capturing dynamic transitions rather than static snapshots:

- Neutral State to Suspicious State: Triggered by anomalous features like rapid limb velocity (greater than threshold derived from pose deltas).
- Suspicious State to Aggressive State: Confirmed by sustained patterns (e.g., raised fists or inter-person proximity less than 0.5m via bounding box overlap).

- Aggressive State to Alert State: Activated when LSTM score greater than 0.85 persists for more than 1 second, initiating alerts and state reset after resolution.
- Guard Conditions: Environmental filters (e.g., crowd density check) prevent transitions in non-hostile contexts.

This FSM integrates with the LSTM for probabilistic state probabilities, enhancing temporal sensitivity.

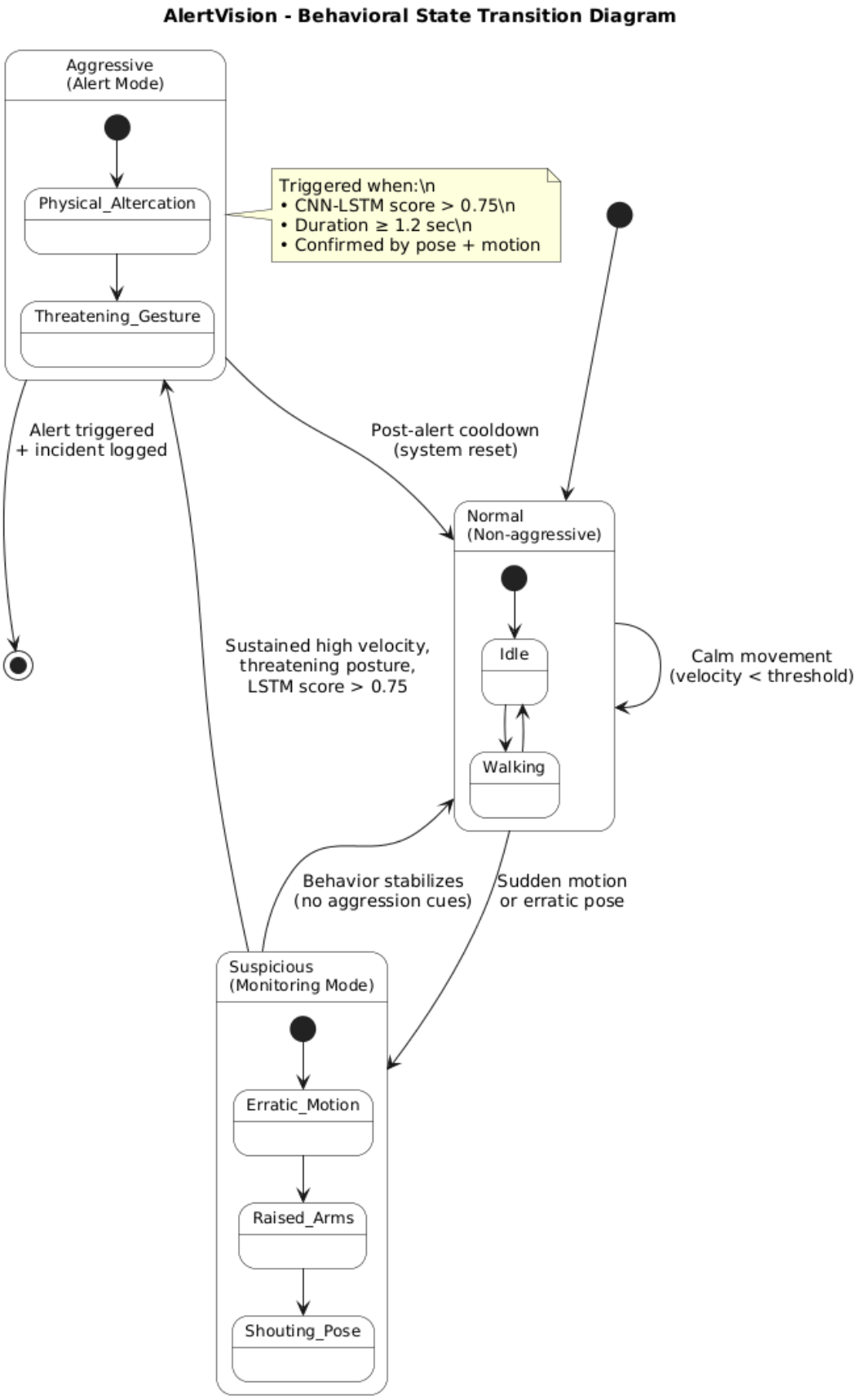

Figure 3.5: Behavioral State Transition Diagram – Temporal Modeling of Aggression States